

FINAL PROJECT - SECURE S3FS

JUSTIN SZERLAG - HK1017

I. Introduction

The initial goal of this project was defined as, “design and implement a stackable file system running on Linux, which uses Amazon’s S3 as backend storage.” This file system had one large criterion though, it has to implement its own local encryption and decryption. The reasoning behind this is quite simple: by encrypting our data before it is sent to Amazon’s cloud service, we no longer put the responsibility of security and data privacy in the hands of Amazon. We reserve the right to know how our data is being encrypted and decrypted, and in the case of any breaches or system failures, we do not stand to lose data to potentially malicious third parties. This filesystem will be created by modifying an open source software called s3fs.

s3fs has many features, but its core functionality is described on its Github page as “[allowing] Linux, macOS, and FreeBSD to mount an S3 bucket via FUSE (Filesystem in Userspace).” Essentially this means s3fs can use Amazon S3 to store your files using cloud storage offered by Amazon, while also allowing you to view, edit, and move files to and from this bucket locally. These changes will be mirrored in the Amazon S3 console as long as the filesystem is mounted correctly.

We will also be crafting our own Advanced Encryption Standard (AES) encryption and decryption tool. This tool will be compatible with other AES implementations of the aes-256-cbc cipher, specifically OpenSSL’s implementation that comes preinstalled on many popular Linux distributions. This standalone tool should also be compatible with the encryption and decryption used by the modified s3fs we plan to implement in this project. This also means that files uploaded to the s3fs system should be able to be decrypted by OpenSSL.

II. Project goals

As stated in the Introduction, the goal of this project is to alter the open source program s3fs, to allow for local encryption and decryption. To achieve this, this project has two key components. The first component is the stand-alone aes-256-cbc program.

The creation of this stand-alone encryption/decryption tool, while not part of the actual filesystem we intend to implement, will serve as a great way to learn and better understand the steps to proper aes-256-cbc encryption. The tool will take a flag to denote encryption or decryption, a password, an input file, an output file, as well as allowing the user to run the encryption with or without salting. By creating this tool and ensuring its compatibility with OpenSSL, we can then expand upon these ideas in the second, and arguably more important portion of this project, the secure s3fs implementation.

As described briefly in the introduction, the secure s3fs is the main goal of this project. This secure implementation of the open source program will utilize similar encryption

procedures when compared to the stand-alone implementation of aes-256-cbc. The files uploaded to the local bucket will be intercepted before they are sent to Amazon S3, encrypted, and then allowed to proceed with the upload to S3. The same will occur for download (read, copy, etc), where the file will be intercepted after it is downloaded from S3, decrypted, and then provided to the location where it is being accessed. This means that applications on computers that have the secure s3fs bucket mounted will be able to modify, read, write, etc. to the file with no explicit decryptions on their end. Encryption and decryption should solely be handled by the s3fs implementation. Another key feature is that any file downloaded from the S3 console, will be downloaded as an encrypted file, and can be decrypted using either OpenSSL or the stand-alone AES implementation covered above.

III. System Information

This project was developed and tested on my personal computer running Windows 11 and using Windows Subsystem for Linux (WSL) to mount and use the S3 bucket. The specification for my system is as follows:

Relevant Hardware:

CPU: AMD Ryzen 7 7800X3D 8-Core Processor, x86_64 architecture

RAM: 2 x 16GB DDR5, 4800 MHz

Storage: 1 TB SSD - Samsung 980

Relevant Software:

Main OS: Windows 11 - Version 24H2

WSL OS: Ubuntu 24.04.3 LTS

FUSE: 2.9.9

IV. Tools and Packages

This project also utilized a variety of tools and packages to allow for proper implementation. All code was written using Visual Studio Code and compiled using GCC inside of WSL.

GCC v.13.3.0 - GCC was used to compile all files related to the stand-alone AES implementation for this project. When compiling using OpenSSL EVP functions, the flags -lcrypto and -lssl were used.

s3fs v1.95 - s3fs was used as the base of the secure s3fs program. This program allows for the mounting of an S3 bucket, and we will be altering the code posted on the most recent github.

OpenSSL v3.0.13 (30 Jan 2024) - OpenSSL and its related headers <openssl/evp.h>, <openssl/rand.h>, and <openssl/err.h> were used inside of both the secure s3fs and stand-alone AES implementation. These allowed for EVP API calls to be used to initialize cipher context and allow for encryption/decryption.

WSL Ubuntu 24.04.3 LTS - Windows Subsystem for Linux was used for this project out of convenience as well as necessity. WSL was great for running Ubuntu terminals and made development and testing efficient.

Other tools and packages:

- Git - Cloning and tracking modifications to the s3fs repository.

- Make - Compiling s3fs using the provided Makefile.

V. Design - 1: 2: s3fs file level vs block level - upload part data flow changing temp file - standalone AES EVP API

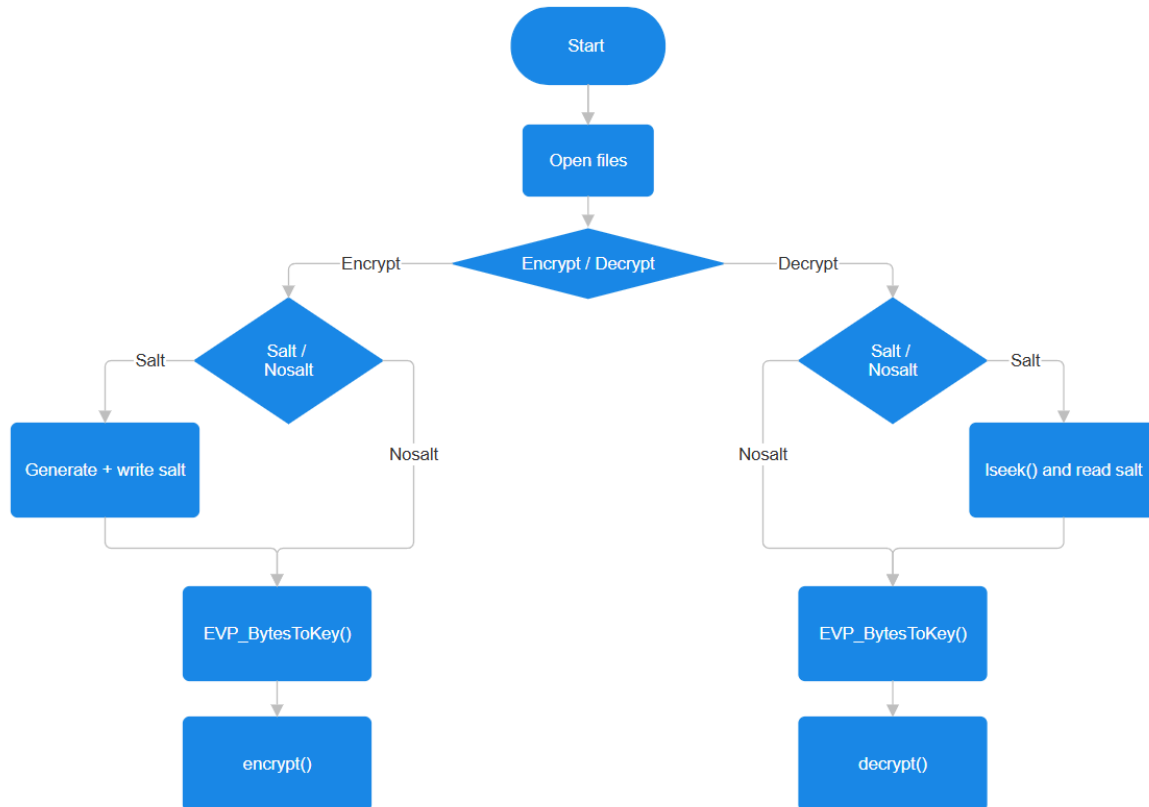
For the first portion of this project, the stand-alone AES tool, the design was fairly simple. The tool must be able to take specific inputs, password, in-file, out-file, encrypt/decrypt, and salt/nosalt. Using these inputs, the tool will encrypt or decrypt a file using aes-256-cbc, and write the output of this encryption (or decryption) to the specified out-file. If the out-file does not exist, a new file will be created in the same directory that the tool is called.

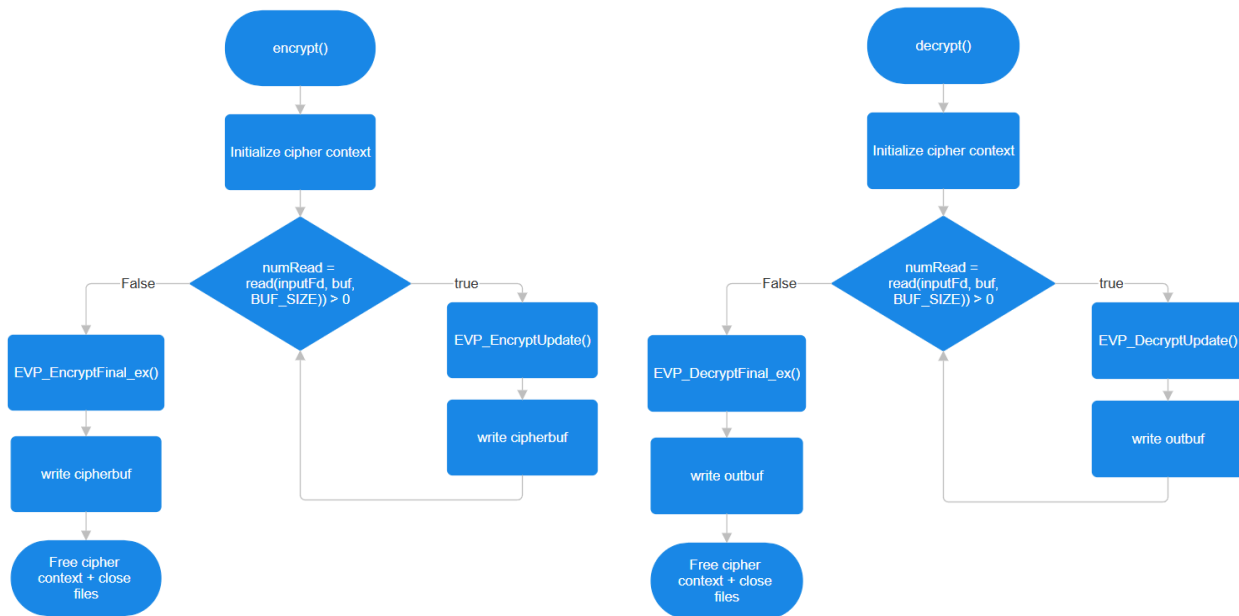
The general flow of encryption and decryption are very similar when using the OpenSSL EVP API calls that we will be utilizing for this project. For both encryption and decryption, there are three main values that we will need the EVP API calls for. These values include, key, initialization vector (IV), and salt. The key acts as the “code” that a cipher will encrypt or decrypt with. Essentially a key acts as the “solution” to a specific set of encrypted or decrypted data, and is needed to correctly decrypt the ciphertext. The way that we can generate a key is by using a password, and an optional salt. Salt acts as another layer of security for encrypted files, making brute force attacks harder. The IV, is a key part of CBC encryption, and ensures different output when encrypting the same file. The IV introduces randomness into the first block of the cipher, ensuring that encrypting the same data will not return the same value, making the cipher more secure. This is because CBC encryption and decryption relies on previous blocks read into the cipher. So essentially the IV randomizes the first block of the cipher, and then the rest of the cipher will be random too, as it depends on the first block, then the second, etc.

The key steps we will need to implement in the AES tool are as follows. Our first step is to receive input from the user (in-file, out-file, password, salt/nosalt), and open the specified files to ensure we can read from the in-file and write to the out-file. We will then decide whether we are encrypting or decrypting. If salt is selected and we are encrypting, we will generate a new eight byte salt using RAND_bytes(). This salt will be written in the format “Salted__(bytes)” at the beginning of the out-file. If we are decrypting and salt is selected, we will seek the position

of the salt, eight bytes from the front of the file, and read it into our salt variable. We will then generate our key and IV using the EVP call `EVP_BytesToKey()`. Here we will enter `EVP_aes_256_cbc()`, `EVP_sha256()` as the first two parameters, and follow this with salt (NULL if nosalt), password, `strlen(password)`, 1, key, IV. This call will fill the key and IV variables with the correct values for our cipher.

Once we have the proper key and iv, we will call either `encrypt()` or `decrypt()`, and in these functions we will begin the cryptography process. The first thing we do is create the cipher context using a pointer to a variable “ctx” of type `EVP_CIPHER_CTX`. We then initialize this context using `EVP_CIPHER_CTX_new()`. Once the context is initialized, we can call `EVPEncryptInit_ex` or `EVP_DecryptInit_ex` respectively. Once these have been completed with no errors, we can read from the in-file into our buffer, and then encrypt or decrypt the data using `EVP_EncryptUpdate` or `EVP_DecryptUpdate` respectively. This new data will be stored in a second buffer, and if we are encrypting, the size of this buffer will be 16 bytes larger than the first buffer to ensure enough room for extra padding the encryption may add. We can then write the encrypted data to the out-file, and repeat the process until we have read the whole file. We are not done yet, we must make sure to call `EVP_EncryptFinal_ex` or `EVP_DecryptFinal_ex` and rewrite the cipher buffer with the new information and padding added by this call. Now all that's left to do is to free the context using `EVP_CIPHER_CTX_free(ctx)`, and close our files and free any allocated data. This process is illustrated by the flow chart below.





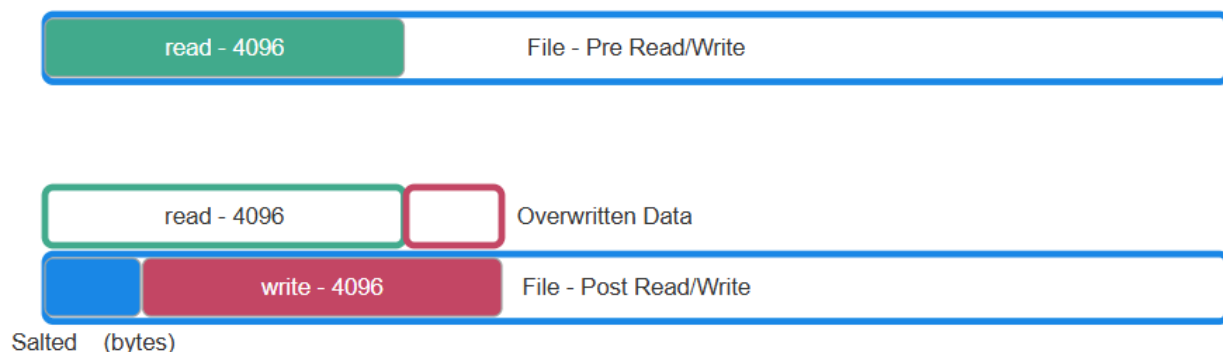
The design of secure s3fs requires that we do more work with the s3fs file system. While the general process of encryption and decryption will stay roughly the same, using the same EVP API calls, there is much more to take into consideration with this portion of the project. The biggest choice I had to make when designing this project was whether the secure s3fs implementation would change data on block level, or file level. Block level alteration of the data would likely mean identifying the components inside s3fs responsible for reading and writing data to temp files, and inserting EVP calls as well as additional logic inside of these components, to encrypt/decrypt data as it is moved to and from S3. The other option for implementing the cryptography for this project was changing data on file level. File level alteration would be less intertwined with the code present inside of s3fs, and would likely only require identifying a proper location for placement of a new encrypt or decrypt function. For encryption, this function would need to be called just before a file is uploaded to S3, but after the user has made any modifications or changed its location in the bucket. For decryption, this function would have to be called after the file is downloaded from S3, but before it is ever shown to the user or any application trying to access it. The process we will be choosing for this particular project will be altering data on the file level, which involves using file descriptors and files already written to a temporary local cache.

Because I have chosen to do file level encryption of data, there are two important factors to getting encryption and decryption to work. These are the creation of the encrypt/decrypt functions, and the placement of their function calls inside of the main s3fs code. We will call

these functions with one parameter, that is the physical file descriptor of the file we want to change.

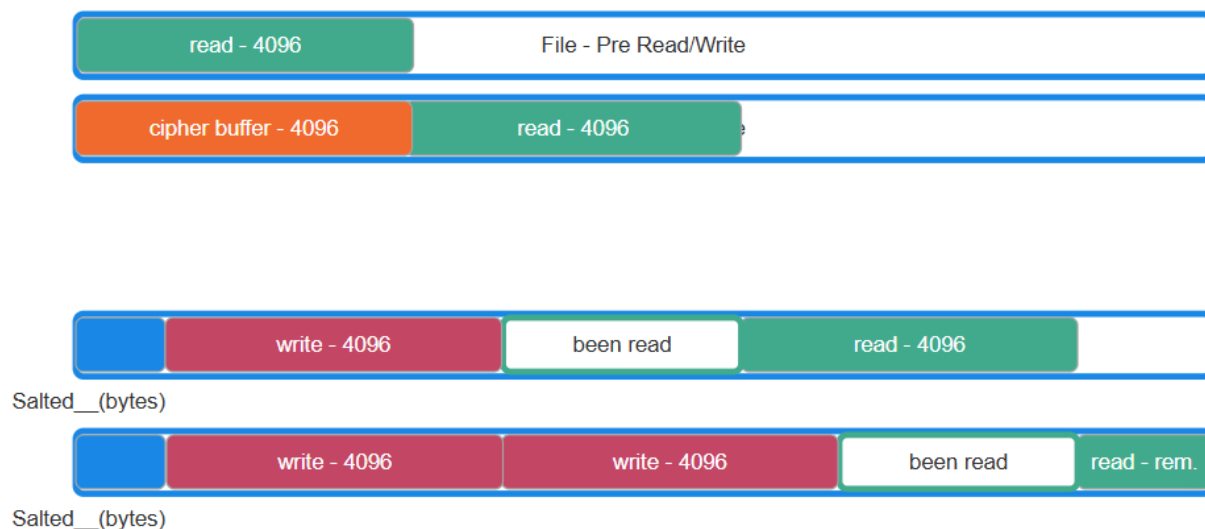
I am going to start by explaining the process of decrypting a file inside of s3fs, as it was the easier portion to implement. The decryption function I have added inside of s3fs is able to decrypt a file without using another temporary file to store data in transit, avoiding having to read and write data twice (once to the temp file, and another back to the original file). This is possible because of the EVP calls requiring two separate buffers. This “in-place” decryption begins similarly to the stand-alone AES, where we will retrieve the salt, create our key and IV, using `EVP_BytesToKey()`, and create/initialize our cipher context. We will then begin decrypting, making sure to track where we are reading/writing from using variables of type `off_t`. The general flow is to read from the file, starting at byte 16 (to avoid reading “Salted__(bytes)”), update our plaintext and store it in our second buffer, and write that buffer from the beginning of the file. This flow ensures that we are never writing over data that has not been read, because we are always reading 16 bytes ahead of our write. Once we have completed our loop, we call the same `EVP_DecryptFinal_ex()` function and write the result of this at the write position we have stored. We can then truncate the file using the write position, as it tracks how many bytes have been written during the process. The last step is to free the cipher context and return from the function. This will essentially decrypt the file in place, minimizing the number of read/writes and squeezing some of the most performance possible out of file level decryption.

Encryption poses a different challenge, as when encrypting a file and using salt, you must write “Salted__(bytes)” at the beginning of the file. The issue arises because if you follow the same idea as the decryption function, tracking the locations of read and write, the read function will always be 16 bytes **behind** the write function. What this means in practice is that if 4096 bytes are read on the first passthrough, the ciphertext that must be written is essentially 4096 bytes plus sixteen bytes for the “Salted__” text. This would overwrite unread data, and cause the encryption to return incorrect, corrupted data.



How we can solve this problem is quite simple in practice. The design of the encrypt function follows a very similar idea to that of the decrypt function, rather we read one buffer ahead of our write. The way this happens is the first buffer will read the plaintext, encrypt it and

store it inside of the cipher-buffer, this buffer will not be written yet. We will then begin our while loop by reading another chunk into the first buffer. Once we have read this chunk, we can now seek to our write position, (originally 0) write our “Salted__(bytes)” and the ciphertext calculated in the first call of `EVP_EncryptUpdate()`. The next step inside of the loop is to update the cipher-buffer, and seek back to our read position where we will read the next block. This will repeat until we reach the end of the file. Once this happens, we will write the remaining data inside of the cipher-buffer, and then continue by calling `EVP_EncryptFinal_ex()` and writing the output of that function to the file. We will then truncate the file for good measure using the write position we have been tracking throughout the function call. Lastly we will free the cipher context. This approach can be seen by the graphic present below.



VI. Integration

For the stand-alone AES I was able to integrate the system using the design described above without any issues. In testing, the stand-alone tool is able to encrypt and decrypt files, with or without salting. The tool can also pair with OpenSSL’s implementation of the aes-256-cbc cipher, and any file encrypted with my tool can be decrypted with OpenSSL, and vice versa. There is one caveat with the current integration of the system however. When decrypting a file that was encrypted with my stand-alone AES using OpenSSL’s aes-256-cbc command, a warning message is displayed which reads as follows: ***** WARNING : deprecated key derivation used.** Using `-iter` or `-pbkdf2` would be better. After doing some research, this warning message is likely because the way that I create my key is somewhat outdated. While the security may be weaker than what is offered today, the AES still works perfectly without using this newer feature, and I’m sure changes could be made to the code to allow for the choice to use this newer key derivation.

For the secure s3fs integration, I would say I had great success integrating with the other tools and libraries I was using throughout this project. Any file that is moved or copied to the

mounted S3 bucket is properly encrypted and sent to the online version of S3. When logging into my bucket using Amazon's S3 console, files can be downloaded in their encrypted version, and decrypted using either the stand-alone AES tool I have worked on throughout this project, or by using OpenSSL's decryption tool. This satisfies the original goal of only encrypted data being stored online, and not letting Amazon be the ones who control my data privacy and security. While not on block level locally, the files are still encrypted before they are sent to Amazon, which I feel is **the most important** aspect of this portion of the project by a large margin. As for decryption using the secure s3fs implementation I have been working on, I have not run into any issues of encrypted files being displayed anywhere after moving, copying, altering, or viewing files stored inside of the mounted bucket. CP works as expected and will decrypt the file before it is moved to the other destination, and commands like cat properly display the original text/data stored inside of the files.

When looking at the secure s3fs implementation, I think what I was able to accomplish was a great, yet very basic implementation of the task at hand. Because I was unable to achieve block level encryption and decryption, the speed of the program likely suffers, as data will be read from S3 to a temp file, and then altered by the encrypt and decrypt function. I have noticed that copying files to S3 can sometimes be slightly delayed, and I assume this is due to the issues mentioned above. Obviously copying the entire data of a file two times is not the ideal solution. Overall, the integration of the two aspects of this project both work, but I do have some worries about the strength of these systems when pushed to their limits.

VII. Implementation Details

We have already explored the key parts that make the stand-alone AES encryption and decryption work in the Design section of this paper. I would like to show these same steps, but relate them to code snippets. For the stand-alone AES the first step is input collection and input validation. In order to facilitate this, I used the supplied file "opt_read.c" with some slight alteration to allow for different types of input. This file gave me the framework for the hand_option function, which validates input. I will also be using global variables to easily set using this function, and for use later in the encrypt and decrypt sections. The first step after reading the user's input, is to open both the input file, and the output file. The input file will be opened as read only, to avoid altering any data inside, the outfile will be opened with read/write permissions. Once we have opened the files, we create our key, iv, and salt variables that will store the respective values and allow us to work with them in the encrypt and decrypt functions. These variables will be of type unsigned char.

Next we will check to see if the user has selected to encrypt or decrypt data. This is done with a simple global variable that is initialized to 1, and changed to 0 if the user has selected decryption. If we are encrypting and we are using salt, a salt will be generated using RAND_bytes(salt, 8) - this will generate a random sequence of 8 bytes and we will fill the salt

pointer with those bytes. If encryption is selected but the user does not want to use salt, then salt will be freed and set to null. If we are decrypting, we instead seek the location of the salt, which is 8 bytes from the beginning, and read that value into the salt variable. Salt will be null if nosalt is used. We can then call our first EVP function, `EVP_BytesToKey()`, which will set our key and IV values.

```
EVP_BytesToKey(EVP_aes_256_cbc(), EVP_sha256(), salt, (unsigned char*)password, strlen(password), 1, key, iv);

encrypt(inputFd, outputFd, key, iv); // OR decrypt(inputFd, outputFd, key, iv);
```

Both the encrypt and decrypt functions are very similar, both begin with context initialization:

```
EVP_CIPHER_CTX *ctx;

if(!(ctx = EVP_CIPHER_CTX_new()))
    handleErrors();

if(1 != EVP_EncryptInit_ex(ctx, EVP_aes_256_cbc(), NULL, key, iv))
    handleErrors();
```

We then enter the while loop where we read and write until the file is empty. Making sure to update the cipher and store it in its buffer after every iteration.

```
while ((numRead = read(inputFd, buf, BUF_SIZE)) > 0) {
    if(1 != EVP_EncryptUpdate(ctx, cipherbuf, &len, buf, numRead)) {
        handleErrors();
    }
    if (write(outputFd, cipherbuf, len) != len) {
        printf("write() returned error or partial write occurred");
        return 1;
    }
    ciphertext_len += len;
}
```

Finally, the last thing we need to do is write the data added by `EVP_EncryptFinal_ex(ctx, cipherbuf, &len)`, and free the cipher context.

```
if(1 != EVP_EncryptFinal_ex(ctx, cipherbuf, &len))
    handleErrors();

if (write(outputFd, cipherbuf, len) != len)
    handleErrors();

EVP_CIPHER_CTX_free(ctx);
```

The work is essentially the same for decryption, just using different decrypt functions instead of encrypt ones, which wraps up the stand-alone AES implementation.

As for the secure s3fs implementation, the decryption function is very similar to the AES implementation. We begin by initializing our variables for key, IV, and salt. We then proceed to

read the salt from the 8th position in the file. The main difference comes before the while loop, and during the loop, where we track the readPos and writePos in order to read and then write to the correct locations in the same file.

```
off_t readPos = 16;
off_t writePos = 0;

while ((numRead = read(fd, buf, BUFF_SIZE)) > 0) {
    readPos += numRead;

    if(1 != EVP_DecryptUpdate(ctx, outbuf, &len, buf, numRead)) {
        printf("EVP_DecryptUpdate() returned error or partial write occurred\n");
        return 1;
    }

    lseek(fd, writePos, SEEK_SET);

    if (write(fd, outbuf, len) != len) {
        printf("write() returned error or partial write occurred\n");
        return 1;
    }

    writePos += len;

    lseek(fd, readPos, SEEK_SET);
}
```

This workflow allows us to avoid using a temporary file and having to read and write two times, once to the temp file, and once back to the main file. As for closing the file and writing the end of the cipher, the same process is followed as the AES.

For Encryption, we differ slightly in that we read two chunks of the main file before writing our first ciphertext block. As explained in the design section, this is to avoid writing over unread data, and harming the accuracy of the encryption. The function begins by initializing cipher context, creating a random salt, and then we begin our reads.

```
// read into buffer
if ((numRead = read(fd, buf, BUFF_SIZE)) < 0) {
    return 1;
}
readPos += numRead;

// fill cipherbuf
if(1 != EVP_EncryptUpdate(ctx, cipherbuf, &len, buf, numRead)) {
    printf("first EVP_EncryptUpdate() returned error or partial write occurred\n");
    return 1;
}
```

Here you can see us reading a chunk and filling the cipher buffer before we begin the while loop. The structure of the while loop can be seen below. It has changed slightly in order of operations to ensure that we are first emptying the filled cipherbuffer, and then creating a new set of ciphertext after the first set has been written.

```

while ((numRead = read(fd, buf, BUFF_SIZE)) > 0) { // read next chunk(s) before overwriting with write - now a
chunk ahead
    readPos += numRead;

    lseek(fd, writePos, SEEK_SET);

    if (writePos == 0) { // if at start of file, write Salted__(salt)
        write(fd, "Salted__", 8);
        write(fd, salt, 8);
        writePos += 16;
    }

    if (write(fd, cipherbuf, len) != len) {
        printf("write() returned error or partial write occurred\n");
        return 1;
    }

    writePos += len;

    if(1 != EVP_EncryptUpdate(ctx, cipherbuf, &len, buf, numRead)) {
        printf("EVP_DecryptUpdate() returned error or partial write occurred\n");
        return 1;
    }

    lseek(fd, readPos, SEEK_SET);
}

```

This while loop is then followed by one last set of writing the cipherbuffer to the writePos of the file. This is done because the while loop, when finished (or if not entered at all), will leave data sitting in the cipher buffer that has not been written. We can then write this data, use our `EVP_EncryptFinal_ex()` function, and write the last amount of data to the file.

```

lseek(fd, writePos, SEEK_SET);
// extra write to ensure all data is written
if (writePos == 0) {
    write(fd, "Salted__", 8);
    write(fd, salt, 8);
    writePos += 16;
}

if (write(fd, cipherbuf, len) != len) {
    printf("write() returned error or partial write occurred\n");
    return 1;
}

writePos += len;

if(1 != EVP_EncryptFinal_ex(ctx, cipherbuf, &len)){
    printf("Error inside EVP_EncryptFinal_ex()\n");
    return 1;
}

if (write(fd, cipherbuf, len) != len){
    printf("Error inside final write()\n");
    return 1;
}

```

As for the placement of these functions in the original s3fs code, the file I have found to work is `fdcache_entity.cpp`. The placement of the decrypt function is inside of the function `FdEntity::Load()` and placed after the file is downloaded from S3. This means its location is just

above the setting of the loaded flag, as close as we can get to the download of the file from S3. The moment the file is accessible in our temp files, we want to change it. The location of the encrypt functions placement is just before the logic that will select the type of upload inside of `FdEntity::RowFlushHasLock()`. This is done before the logic that selects whether it is a multipart upload, streamupload, etc to ensure that all upload types are still just using the encrypted file. We want to encrypt it before these uploads begin and this was the location I found to do that.

VIII. Future Improvement

My current implementation of the secure s3fs is by no means absolutely complete. There are many things that, given enough time and thought, could be changed. I think the largest and most influential change that I would make, given enough time, would be to swap to block level encryption. I feel like I may have found one of the spots for this encryption earlier in my Logs> I have reason to believe `curl.cpp` has a lot of the buffers and other components necessary to implement block level. Block level would likely speed up the processing of the files, and be better intertwined with the s3fs filesystem as a whole. The main idea behind how I think block level is implemented, revolved around finding the buffers that s3fs uses to read data into the temp files, and adding EVP calls as well as extra logic in these locations to allow for encryption AS the bytes are read and before the are written to their destination. I was unable to identify these locations that I know of. I think with what I know now after completing the file level version of this project, I am capable of doing block level, again, I may just need more time.

There is also an issue that has/can occur when creating files using nano inside of the s3 bucket. After implementing the ability to read the password from a password file, some issues were arising with decryption not working correctly on small files created with nano. I was unable to completely resolve this issue, and I'm not sure what the cause could be.

I would say most of the future improvements I would like to make are related to block level, I was able to make some very drastic improvements upon my first iteration of the encrypt and decrypt functions inside of secure s3fs. The first iteration of these functions were much more primitive, essentially reading data, writing the encrypted/decrypted data to a temporary file, and then re-reading and re-writing the temporary file data back to the original file. This was quite slow, and clearly not the most efficient way to go about the data transfer. The decrypt function that I was able to make was much easier, but I had to take a step away from the project and brainstorm how to solve the encrypt function, which I am proud I was able to do in a quicker way.

Another thought I had about the efficiency of not only the s3fs implementation, but the stand-alone AES, was that if encryption was faster than read/write (especially if using an HDD) then efficiency could be sped up using a circular queue of buffers similar to the last Lab for this class. My thought was that if encrypting is, say, 5 times faster than read or write, then you could create ~5 threads with the sole purpose of reading into a circular buffer implementation. This implementation is something I have used for a couple labs, but the last long lab is the one I

would've tested. I was unable to find the time to test if encryption was actually faster than read or write, especially since I am using a SSD, I was less worried about it in my case. I think this test could be run using some of the `clock_gettime` functions we have been using throughout the semester. I would be very interested to see if that would've made a difference in efficiency, especially when encrypting or decrypting very large files.

As far as any more improvements, I'm sure if I really analyze my code I will find all sorts of little things I will want to tinker with and change, but as for the core functionality of the code, I am quite happy.

IX. Summary

Overall, I would say this project was challenging, but has made me better in a lot of ways. One of the biggest things that this project forced me to do was read a lot of documentation. Thankfully a lot of what we actually needed was supplied to us, but I haven't had to read that much documentation for any project I've worked on. I feel like I've learned a lot about the steps to cryptography, the flow of data in a filesystem like s3fs, and I've learned a lot about the structure of code in a huge codebase. This is my first time working with a codebase the size of s3fs, and the task of finding *anything* inside of the codebase was incredibly daunting at the start of this project. As we have gone on I feel like I am better at analysing code structure, and I've gotten a sneak peak at what working on a large codebase truly looks and feels like.

This class has really made me feel more like a genuine programmer. Learning about threads, processes, inter-process communication, and every aspect covered in each of the lectures, has been really interesting. I will say there were a lot of hiccups with this class period, but I don't think those tribulations stopped me from learning really valuable information, and applying very important skills. I really enjoyed the Labs that were offered, but I wish there was slightly more information presented about the project. Sometimes it could feel very daunting and like I was missing a LOT of information.

I will say that I am very proud of myself, for getting through this project. I have had a very rough year and this class tested me at points, but I held strong and was able to deliver what I feel is quality work and submissions. I feel like I have really learned a lot of aspects of low level programming and I feel like I really applied myself during this semester, even when times were tough. I am happy to be submitting this project, and I feel good about the work that I have done.

References: Resources and code snippets that helped me complete this project

<https://github.com/s3fs-fuse/s3fs-fuse>

<https://en.wikipedia.org/wiki/PBKDF2#:~:text=PBKDF2%20applies%20a%20pseudorandom%20function,cryptographic%20key%20in%20subsequent%20operations.>

<https://wiki.openssl.org/index.php/EVP>

https://wiki.openssl.org/index.php/EVP_Symmetric_Encryption_and_Decryption

<https://eclipsesource.com/blogs/2017/01/17/tutorial-aes-encryption-and-decryption-with-openssl/>

https://docs.openssl.org/3.5/man3/EVP_BytesToKey/

Supplied files: copy.c, opt_read.c